

Python Decorators Cheat Sheet

A quick reference guide for commonly used function, class, and method decorators in Python.

Basic Decorator

Wraps a function with additional behavior that can be reused. Allows the function to focus on core behavior and the decorator to provide a useful enhancement to the decorated function.

```
def my_decorator(func):
    def wrapper():
        print('Before')
        func()
        print('After')
    return wrapper

@my_decorator
def say_hello():
    print('Hello')
```

Decorators with Arguments

Decorators with arguments allow some additional configuration to the behavior of the decorator

```
import functools

def repeat(n):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for _ in range(n):
                func(*args, **kwargs)
        return wrapper
    return decorator

@repeat(3)
def greet(name):
    print(f'Hello {name}')
```

Function Decorators

- `@functools.wraps(func)`

Preserves the original function's metadata (like name and docstring) when wrapping it with a decorator.

Example:

```
import functools

def log_calls(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        print(f'Calling {func.__name__}')
        return func(*args, **kwargs)
    return wrapper
```

- `@functools.lru_cache(maxsize=None)`

Caches function results to speed up repeated calls with the same arguments.

Example:

```
from functools import lru_cache

@lru_cache(maxsize=128)
def fibonacci(n):
    if n < 2:
        return n
    return fibonacci(n-1) + fibonacci(n-2)
```

Method Decorators

- `@staticmethod`

Defines a method that does not operate on an instance or class. Behaves like a normal function inside a class.

Example:

```
class MathUtils:
    @staticmethod
```

```
def add(a, b):  
    return a + b
```

- `@classmethod`

Defines a method that receives the class (`cls`) as its first argument instead of `self`.

Example:

```
class Example:  
    count = 0  
  
    @classmethod  
    def increment(cls):  
        cls.count += 1
```

- `@property`

Converts a method into a read-only attribute.

Example:

```
class Circle:  
    def __init__(self, radius):  
        self._radius = radius  
  
    @property  
    def area(self):  
        return 3.14 * self._radius ** 2
```

Class Decorators

- `@dataclass`

Automatically generates `__init__`, `__repr__`, and `__eq__` methods based on class fields.

Example:

```
from dataclasses import dataclass  
  
@dataclass  
class Point:
```

```
x: int
y: int
```

- `@abc.abstractmethod`

Marks a method as abstract within an abstract base class. Subclasses must implement it.

Example:

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass
```

Custom Decorators

- Example: Logging function calls

```
def log_calls(func):
    def wrapper(*args, **kwargs):
        print(f'Calling {func.__name__}')
        result = func(*args, **kwargs)
        print(f'{func.__name__} returned {result}')
        return result
    return wrapper

@log_calls
def add(a, b):
    return a + b
```

Notes

- Decorators can be stacked: `@A then @B` means `A ( B ( func ) )`.
- Always use `@functools.wraps` for custom decorators.
- Class decorators take a class and return a class.
- Method decorators modify behavior within classes.